PROBLEM

Your camera roll is a graveyard of intentions.

We screenshot concert flyers, products, books, places and charts meaning to act — then they're buried forever. Nothing turns a screenshot into the next step.

Cloud screen assistants can't touch a work device

"Read my screen" tools ship pixels to someone else's servers. On devices under NDAs, SOC 2 / HIPAA / GDPR duties, that's a non-starter — the useful screenshots are exactly the ones you can't upload.

So the intent dies

No safe path from screenshot → action means people fall back to manual copy-paste, or just forget. The value is stranded on-device.

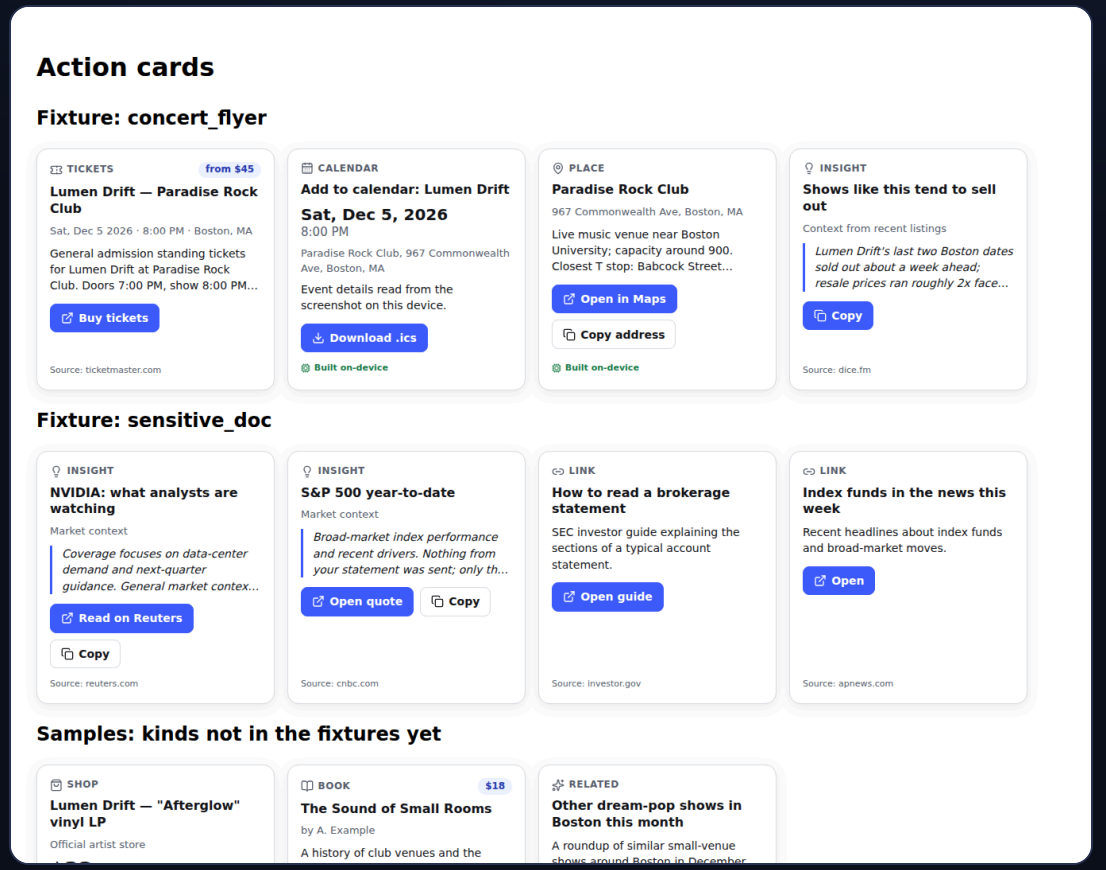
LATENT

Drop a screenshot. Get actions — privately.

Latent is a local-first memory agent: a screenshot is understood by a vision model **on your machine**, private data is scrubbed, and only short text queries leave — blooming into action cards you can act on.

> No image or raw text leaves the device — enforced by code and shown in the audit log.

→ Hand off to the live demo



Action cards (fixture gallery). Swap for a bloom still once the WebGL dashboard (#13) lands.

ARCHITECTURE

One door to the network.

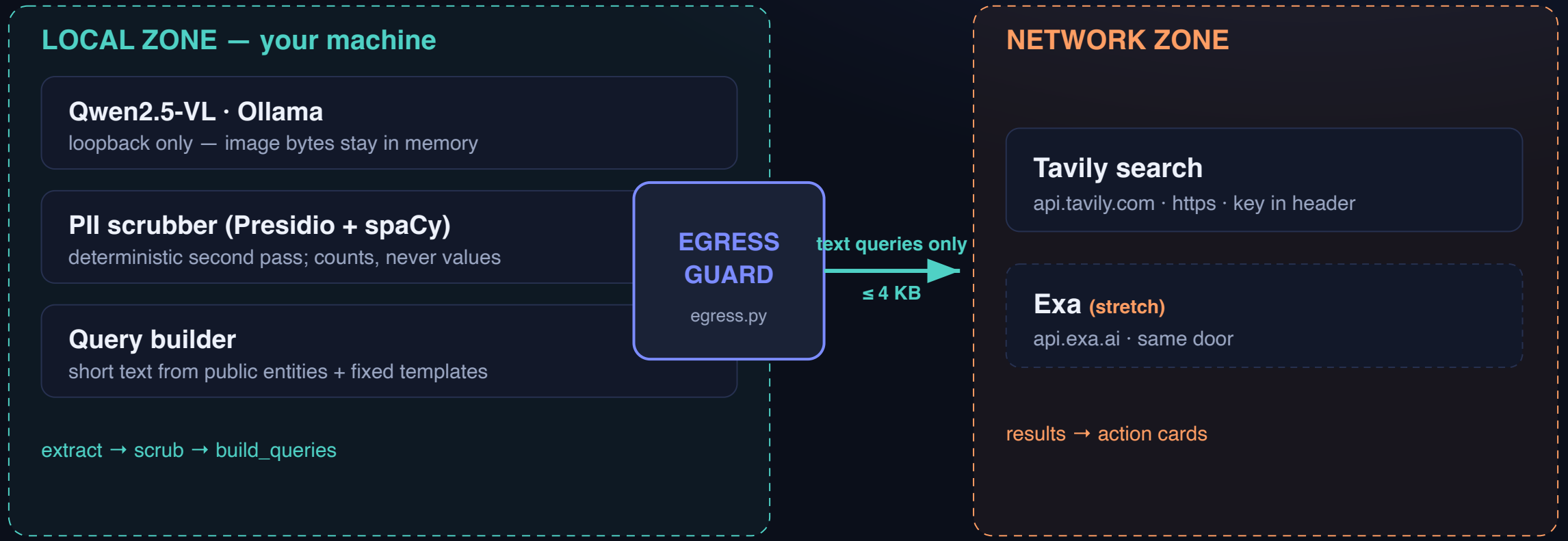


Image bytes never cross the dashed local boundary. Ollama is reached only via `egress.local_client()` (loopback). Every outbound request passes `egress.internet_post`.

PROOF

Enforced by code, measured by eval.

10	—%	0	—s
synthetic screenshots (planted fake PII)	intent accuracy *[run before demo]*	leaks target *[confirm in results.md]*	latency p50 *[run before demo]*

Eval numbers are placeholders — **[from `eval/results.md` — run `LATENT_OFFLINE=1 python eval/run_eval.py` before the demo]**. Leak = any planted `must_not_appear` string in the scrubbed Intent or any query.

The six egress rules (`egress.py`)

1. Air-gap: `LATENT_OFFLINE=1` blocks everything
2. Allow-list host, https, default port
3. Body ≤ 4096 bytes (no image fits)
4. No base64-like blob (200+ chars)
5. PII check finds nothing in the body
6. No API key in body (headers only)

Air-gap demo

Flip the air-gap toggle, drop an internal email: extraction still finishes `LOCAL`, search is blocked `NET ×`. The audit log shows the attempt, its byte count and the reason.

BUSINESS MODEL

Privacy is the product — and the moat.

B2C — affiliate share

Action cards link to tickets, shopping and books. Every commercial URL already routes through the `affiliate.tag_url` hook in `cards.py` — the single point where a partner tag is appended for a referral share. Free to the user; revenue on intent we already surfaced.

The hook ships wired into every card; it returns the URL unchanged until a partner tag is set.

B2B — enterprise tier

The exact thing that blocks cloud assistants is our sale:

- On-prem / fully local model
- Policy controls over what may leave
- Audit-log export for compliance

Same engine, two doors: consumers get convenience, enterprises get provable control.

TRACKS & ASK

Three tracks, one build.

- **Privacy-First (main):** single egress door, six enforced rules, air-gap toggle, visible audit log.
- **SURGE (Tavily & Exa):** sanitized text queries drive Tavily today; Exa through the same door as a stretch.
- **Most Startup-able:** B2C affiliate + B2B on-prem tier on one engine.

What's next

- WebGL bloom dashboard (#13)
- Exa search through the same door
- Run the eval, publish the numbers

Team

[Team names]

> The ask: back the local-first agent that turns stranded screenshots into action — without the pixels ever leaving the device.